

CIVILIZATIONS

Manual de Documentación Técnica del Proyecto

Proyecto AWS — Curso 25/26



Autores

Aitor Mera · Miguel Ángel Galdón · Styven Catagua

Índice

1. Introducción del proyecto	3
1.1. Descripción	3
1.2. Objetivos	3
1.3. Herramientas utilizadas	3
2. Análisis y diseño	4
2.1. Especificaciones funcionales	4
2.2. Especificaciones no funcionales	4
2.3. Estructura y estados del proyecto	4
2.4. Mecánicas principales	6
2.5. Diagramas UML	6
3. Arquitectura y desarrollo	7
3.1. Estructura del código fuente	7
3.2. Componentes principales (adaptado al juego)	7
3.3. Diagrama de ficheros del proyecto	8
3.4. Flujo de datos y eventos	8
4. Test y depuración	9
4.1. Pruebas realizadas	9
4.2. Errores encontrados y solución	9
4.3. Mejoras aplicadas durante el desarrollo	9
5. Recursos gráficos y de audio	10
6. Manual de usuario	11
6.1. Requisitos e instalación	11
6.2. Objetivo del juego	11
6.3. Controles (interfaz gráfica)	12
6.4. Ejemplo de partida	13
7. Reflexión y mejoras	13
7.1. Dificultades y cómo se superaron	13
7.2. Mejoras futuras	13
7.3. Reparto de tareas	13

1. Introducción del proyecto

1.1. Descripción

Civilizations es una aplicación de gestión y estrategia por turnos en la que el jugador dirige una pequeña civilización. El jugador genera recursos, construye edificios, mejora tecnologías y recluta unidades militares (de ataque, de defensa y especiales) para defenderse de un ejército enemigo que ataca de forma periódica y automática.

El juego no es de acción en tiempo real: no existen colisiones ni proyectiles. El combate se resuelve mediante una simulación automática por turnos en la que cada unidad ataca según unas probabilidades configurables. Toda la partida (recursos, ejército, edificios, tecnologías e historial de batallas) se persiste en una base de datos para poder reanudarla y para alimentar la web informativa del módulo M04.

1.2. Objetivos

- Implementar en Java un modelo de objetos completo (herencia, clases abstractas, interfaces y polimorfismo) que represente la civilización, las unidades y las batallas.
- Simular batallas automáticas equilibradas mediante un sistema de probabilidades parametrizable.
- Persistir el estado del juego y el historial de batallas en una base de datos.
- Ofrecer una interfaz gráfica de usuario (Swing) además de la versión por consola.
- Publicar una web (módulo M04) con información actualizada desde la base de datos.

1.3. Herramientas utilizadas

Categoría	Herramienta
Lenguaje	Java 17
IDE	Eclipse
Interfaz gráfica	Java Swing
Base de datos	MySQL
Web (M04)	Node.js + Express + plantillas HBS
Control de versiones	Git y GitHub
Diagramas UML	Herramienta de modelado (M05)

2. Análisis y diseño

2.1. Especificaciones funcionales

- Generación periódica de recursos (comida, madera, hierro y maná) mediante una tarea programada que se ejecuta cada cierto tiempo.
- Construcción de edificios (granja, carpintería, herrería, torre mágica e iglesia) que incrementan la generación de recursos o habilitan unidades especiales.
- Mejora de tecnologías de ataque y defensa, con coste creciente por nivel.
- Reclutamiento de nueve tipos de unidad: cuatro de ataque, tres de defensa y dos especiales (mago y sacerdote).
- Generación automática de un ejército enemigo cada cierto tiempo, cuyo tamaño crece con el número de batallas jugadas.
- Resolución automática de la batalla y generación de un informe paso a paso y un informe resumen.
- Persistencia del estado y del historial de batallas en base de datos.

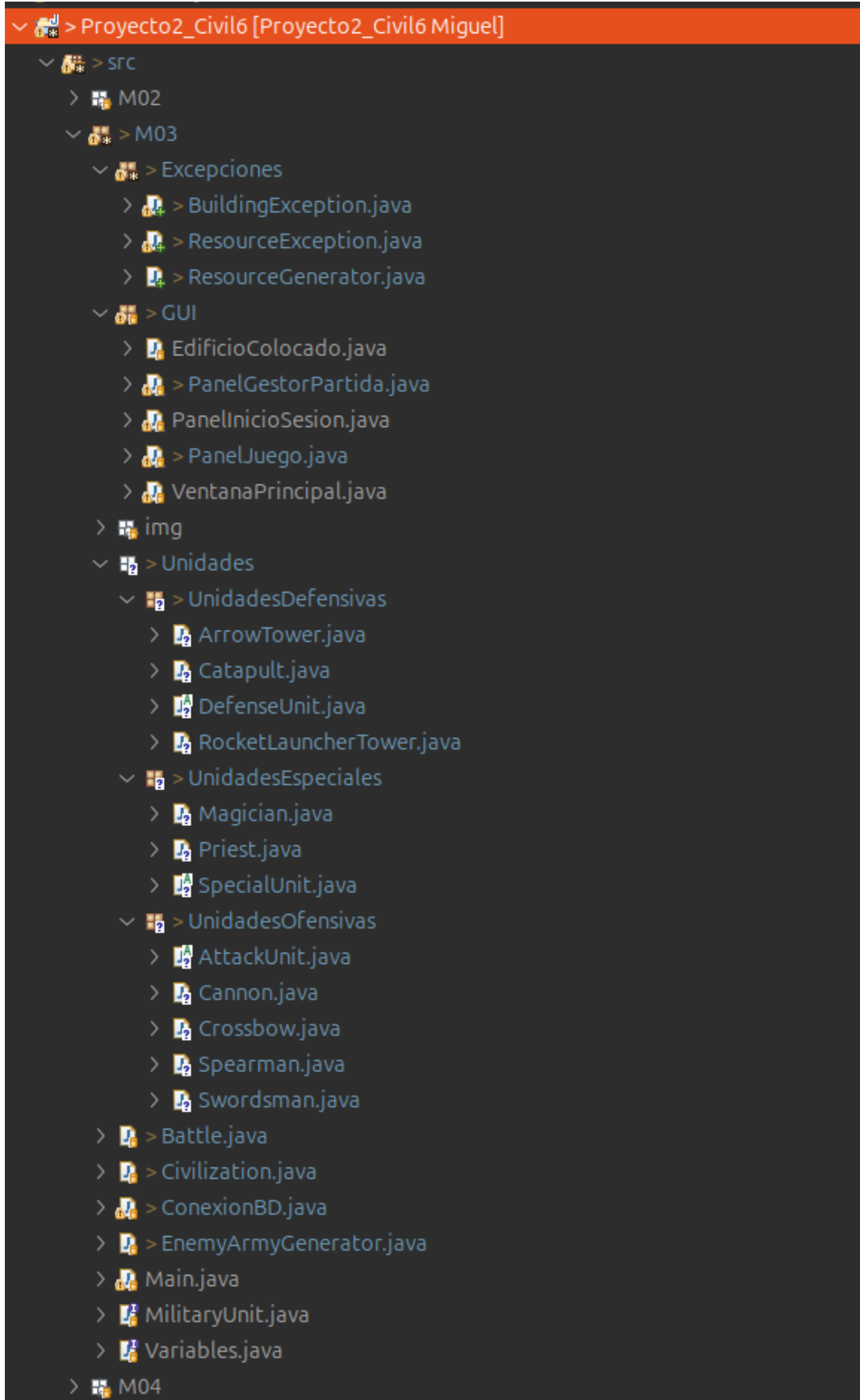
2.2. Especificaciones no funcionales

- La aplicación debe poder ejecutarse tanto por consola como mediante interfaz gráfica.
- Los valores que afectan al equilibrio del juego (costes, probabilidades, tiempos) están centralizados en una única interfaz de constantes para poder ajustarlos sin tocar la lógica.
- La web del M04 debe mostrar datos actualizados procedentes de la misma base de datos.
- El código respeta convenciones estrictas: todos los atributos privados, sin uso de expresiones lambda, sin switch/case.

2.3. Estructura y estados del proyecto

La aplicación se organiza en torno a tres estados principales que el jugador percibe durante la partida:

1. Gestión: el jugador consulta su estado y decide en qué invertir los recursos (edificios, tecnologías, unidades).
2. Amenaza: se ha generado un ejército enemigo y el jugador puede consultar su composición antes del combate.
3. Batalla e informe: el combate se resuelve automáticamente y se muestra el resultado; el estado se guarda en la base de datos.



2.4. Mecánicas principales

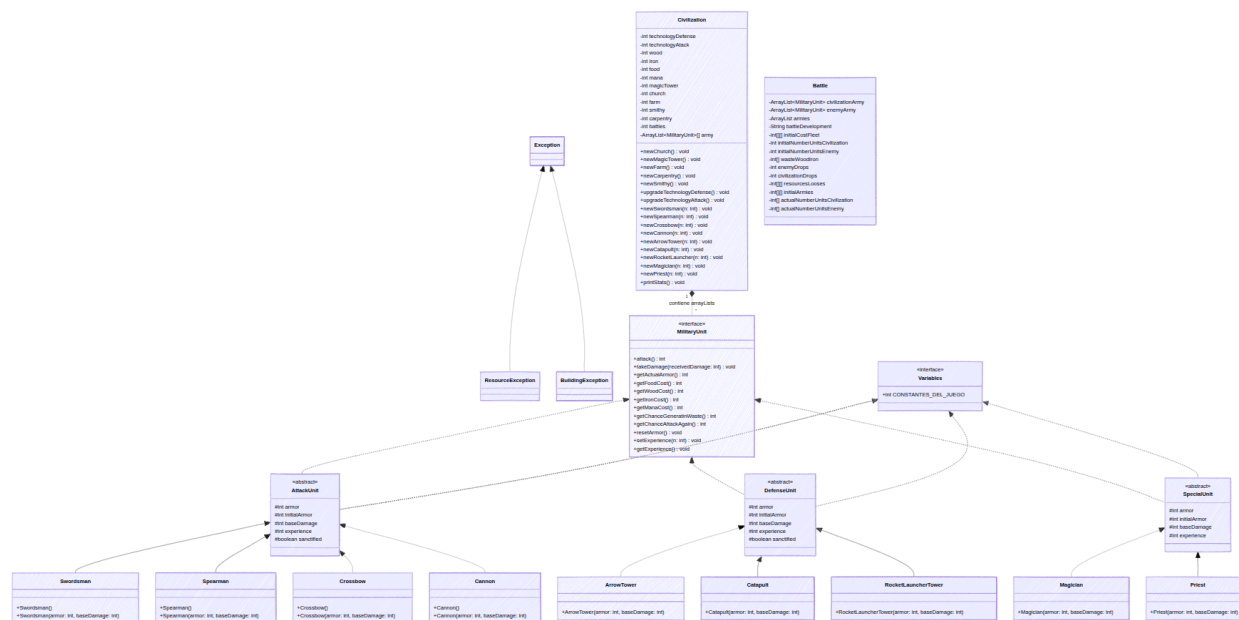
Recursos: la civilización genera una cantidad base de comida, madera y hierro en cada ciclo. Los edificios añaden un porcentaje extra de generación. El maná solo se genera si existe al menos una torre mágica.

Unidades: cada unidad tiene armadura, daño base, experiencia y, en el caso de las unidades de ataque y defensa, un estado de santificación. La armadura y el daño dependen del nivel de tecnología en el momento de crear la unidad. Las unidades supervivientes ganan un punto de experiencia tras cada batalla.

Batalla: comienza atacando un ejército elegido al azar. En cada turno se selecciona un grupo atacante según unas probabilidades fijas, una unidad de ese grupo al azar, y un grupo defensor proporcional al número de unidades vivas. La unidad atacante inflige su daño; si la armadura del defensor llega a cero o menos, se elimina y puede generar escombros (madera y hierro). La unidad atacante puede repetir ataque con cierta probabilidad. La batalla termina cuando uno de los ejércitos baja del 20 % de sus unidades iniciales; el ganador se decide por las pérdidas ponderadas de recursos.

2.5. Diagramas UML

El diagrama de clases recoge las relaciones de herencia entre la interfaz `MilitaryUnit`, las clases abstractas (`AttackUnit`, `DefenseUnit`, `SpecialUnit`) y las nueve clases concretas, así como las clases `Civilization`, `Battle` y las excepciones propias.



3. Arquitectura y desarrollo

3.1. Estructura del código fuente

Todo el código del módulo M03 se encuentra en el paquete M03. Los componentes principales son:

Componente	Responsabilidad
Variables (interfaz)	Todas las constantes del juego: costes, probabilidades, tiempos, daños y armaduras.
Civilization	Estado del jugador: recursos, edificios, tecnologías y ejército. Métodos para construir, mejorar y reclutar.
MilitaryUnit (interfaz)	Contrato común a todas las unidades (atacar, recibir daño, costes, etc.).
AttackUnit / DefenseUnit / SpecialUnit	Clases abstractas con la lógica común de cada familia de unidades.
Swordsman, Spearman, Crossbow, Cannon, ArrowTower, Catapult, RocketLauncherTower, Magician, Priest	Las nueve unidades concretas.
ResourceException / BuildingException	Excepciones propias para recursos o edificios insuficientes.
ResourceGenerator	Tarea programada que genera recursos periódicamente.
EnemyArmyGenerator	Tarea programada que crea el ejército enemigo y lanza la batalla.
Battle	Toda la mecánica de combate y la generación de informes.
Main	Punto de entrada, menú de consola y planificación de tareas con Timer.

3.2. Componentes principales (adaptado al juego)

El documento de referencia menciona componentes típicos de un juego de acción (jugador, enemigos, colisiones, proyectiles, HUD). Civilizations es un juego de gestión por turnos, por lo que esos componentes se traducen así:

Componente genérico	Equivalente en Civilizations
Jugador	Clase Civilization (recursos, edificios, tecnologías, ejército).
Enemigos	Ejército generado por EnemyArmyGenerator.
Colisiones	No aplica. El combate es por turnos; el contacto se resuelve por selección de atacante/defensor.
Proyectiles	No aplica. El daño se calcula directamente con el método attack() de cada unidad.
HUD	Salida por consola (printStats) e interfaz gráfica Swing con paneles de recursos, edificios y batallas.

3.3. Diagrama de ficheros del proyecto

La estructura del repositorio sigue la organización por módulos exigida:

```
Civilizations/  
  M01/  Manual y documentación  
  M02/  Scripts SQL de la base de datos  
  M03/  Código Java (paquete M03)  
  M04/  Web Node.js + Express + HBS  
  M05/  Diagrama de clases y README
```

3.4. Flujo de datos y eventos

El programa principal arranca la conexión a la base de datos y planifica dos tareas con un objeto Timer: una que genera recursos periódicamente y otra que, cada cierto tiempo, crea el ejército enemigo, ejecuta la batalla y guarda el resultado.

1. El Timer dispara ResourceGenerator: se suman los recursos generados al estado de la civilización y se guarda el cambio.
2. El Timer dispara EnemyArmyGenerator: se crea el ejército enemigo y se muestra la amenaza.
3. Se crea un objeto Battle con el ejército del jugador y el enemigo; el constructor prepara las estructuras internas.
4. Se llama al método de combate, que resuelve la batalla turno a turno y registra todo el desarrollo.
5. Al terminar, se actualizan la experiencia y los recursos, se sincroniza el ejército del jugador y se guarda la batalla en la base de datos.
6. La web del M04 lee esos datos para mostrarlos actualizados.

4. Test y depuración

4.1. Pruebas realizadas

- Pruebas manuales de generación de recursos con distinto número de edificios.
- Pruebas de creación de unidades verificando que se lanza `ResourceException` o `BuildingException` cuando faltan recursos o edificios.
- Pruebas del generador de ejército enemigo con distinto número de batallas previas, comprobando que el ejército crece de forma coherente.
- Pruebas de la clase `Battle` ejecutando batallas completas y verificando manualmente los costes, las pérdidas, los escombros y el ganador con cálculos hechos a mano.
- Comprobación de que las unidades muertas se eliminan correctamente tanto de la batalla como del ejército real de la civilización.

4.2. Errores encontrados y solución

Error detectado	Solución aplicada
El informe mostraba el mensaje “CHANGE ATTACKER” repetido sin combate cuando el grupo elegido estaba vacío.	Se movió el registro del mensaje para que solo se escriba cuando realmente hay un ataque.
El atacante perdía su turno cuando el algoritmo de probabilidades elegía un grupo propio vacío.	Se eliminó el cambio de bando en ese caso; el atacante mantiene el turno y se vuelve a elegir grupo.
Los costes y la generación del enunciado hacían el juego inviable (varios minutos para una sola unidad).	Se reajustaron los costes a la mitad, se aceleró la generación a 30 s y se redujeron los recursos iniciales del enemigo.
El mago y el cañón repetían ataque con probabilidad tan alta que aniquilaban ejércitos enteros en un turno.	Se redujeron las probabilidades de repetición de mago, cañón y ballesta para equilibrar las batallas.

4.3. Mejoras aplicadas durante el desarrollo

- Centralización de todos los parámetros de equilibrio en la interfaz `Variables` para poder ajustar el juego sin tocar la lógica.
- Reajuste completo del balance económico y de combate tras las primeras pruebas de juego.
- Separación clara entre la preparación de la batalla (constructor) y su ejecución (método de combate).

5. Recursos gráficos y de audio

Al ser una aplicación de gestión, el juego no utiliza audio. Los recursos gráficos se emplean en la interfaz Swing y, sobre todo, en la web del M04, donde se usan iconos para representar los recursos (comida, madera, hierro y maná) y el logotipo del juego, manteniendo una estética común entre la aplicación y la web.

- Logotipo del juego: usado en la portada de este manual y en la cabecera de la web.
- Iconos de recursos: usados en los informes de batalla de la web para representar comida, madera, hierro y maná.

Las imágenes utilizadas son de elaboración con ia o de bancos de recursos de uso libre, integradas en la interfaz gráfica y en las plantillas HBS de la web.



aitor.png



carpinteria.png



fondo_partida.jpeg



granja.png



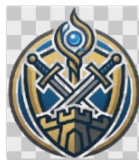
herreria.png



iglesia.png



logo.jpg



logo.png



miguel.png



pantalla_inicial.png



partida.png



styven.png



torreMagica.png

6. Manual de usuario

6.1. Requisitos e instalación

1. Tener instalado Java (versión 17 o superior) y un servidor MySQL en marcha.
2. Importar los scripts SQL de la carpeta M02 para crear la base de datos.
3. Abrir el proyecto de la carpeta M03 en Eclipse.
4. Configurar los datos de conexión a la base de datos (usuario y contraseña) si fuese necesario.
5. Ejecutar la clase Main para arrancar el juego.

6.2. Objetivo del juego

El objetivo es mantener y hacer crecer la civilización resistiendo los ataques periódicos del ejército enemigo. Cada batalla la gana el ejército que sufre menos pérdidas ponderadas de recursos. Si la civilización gana, recoge los escombros de madera y hierro generados durante el combate. El enemigo se vuelve más fuerte con cada batalla, por lo que el jugador debe invertir bien sus recursos en edificios, tecnologías y unidades.

6.3. Controles (interfaz gráfica)

La interfaz gráfica Swing ofrece las mismas funciones que la consola mediante paneles y botones: un panel de recursos, un panel para construir edificios y mejorar tecnologías, un panel para reclutar unidades y una zona donde se muestran los informes de batalla.





```

===== RESUMEN FINAL Y ESTADÍSTICAS =====

BATTLE NUMBER: 3
BATTLE STATISTICS
Army planet      Units  Drops  Initial Army Enemy  Units  Drops
Swordsman        12      0      Swordsman           0      3
Spearman          3      0      Spearman            0      2
Crossbow         24      0      Crossbow            0      0
Cannon            0      0      Cannon              0      0
Arrow Tower       0      0
Catapult          2      0
Rocket Launcher   0      0
Magician          0      0
Priest            0      0
*****
Cost Army Civilization      Cost Army Enemy
Food: 55500                  Food: 17000
Wood: 511000                 Wood: 10500
Iron: 84875                   Iron: 125
*****
Losses Army Civilization      Losses Army Enemy
Food: 0                       Food: 17000
Wood: 0                       Wood: 10500
Iron: 0                       Iron: 125
*****
Waste Generated:
Wood 6300
Iron 68
Battle Wonned by Civilization, We Collect Rubble
#####

```

6.4. Ejemplo de partida

1. Al arrancar, esperar al primer ciclo de generación de recursos.
2. Construir una granja, una carpintería y una herrería para aumentar la generación.
3. Reclutar unidades de ataque y defensa con los recursos disponibles.
4. Antes de la batalla, consultar la amenaza enemiga para decidir si reforzar el ejército.
5. Tras la batalla, revisar el informe y, si se ha ganado, comprobar los escombros recogidos.
6. Guardar la partida y repetir el ciclo, mejorando tecnologías a medida que el enemigo se vuelve más fuerte.

7. Reflexión y mejoras

7.1. Dificultades y cómo se superaron

La parte más compleja fue la clase Battle, por la cantidad de estructuras de datos que coordina (el ejército del jugador y el enemigo en listas planas y clasificadas por tipo a la vez) y por la mecánica de turnos con repeticiones de ataque y eventos especiales. Se resolvió dividiendo el trabajo en fases pequeñas y verificables y comprobando los cálculos a mano tras cada batalla de prueba.

El equilibrio del juego también supuso un reto: con los valores del enunciado era prácticamente imposible jugar. Se solucionó centralizando todas las constantes y reajustando costes, tiempos y probabilidades de forma iterativa hasta conseguir partidas jugables.

7.2. Mejoras futuras

- Implementar la resurrección de unidades por parte de los magos (la constante ya existe pero no se usa).
- Añadir un coste por reiniciar la armadura del ejército tras cada batalla.
- Hacer que el intervalo entre ataques sea aleatorio en lugar de fijo.
- Limitar el nivel máximo de las tecnologías.
- Añadir mantenimiento de comida para algunas unidades, que mueran si no se pueden alimentar.

7.3. Reparto de tareas

Integrante	Tareas
Aitor Mera	Programación interna: lógica del código, clases del modelo (unidades, Civilization, Battle), excepciones y mecánica de juego.
Miguel Ángel Galdón	Desarrollo del juego e interfaz gráfica Swing y creación de la BD.
Styven Catagua	Desarrollo de la página web del módulo M04.